

Excel Shrink: Reducing File Bloat in Destatis Excel Spreadsheets by Up to 99.99%

Anonymous Author(s)
Submission Id: 1503

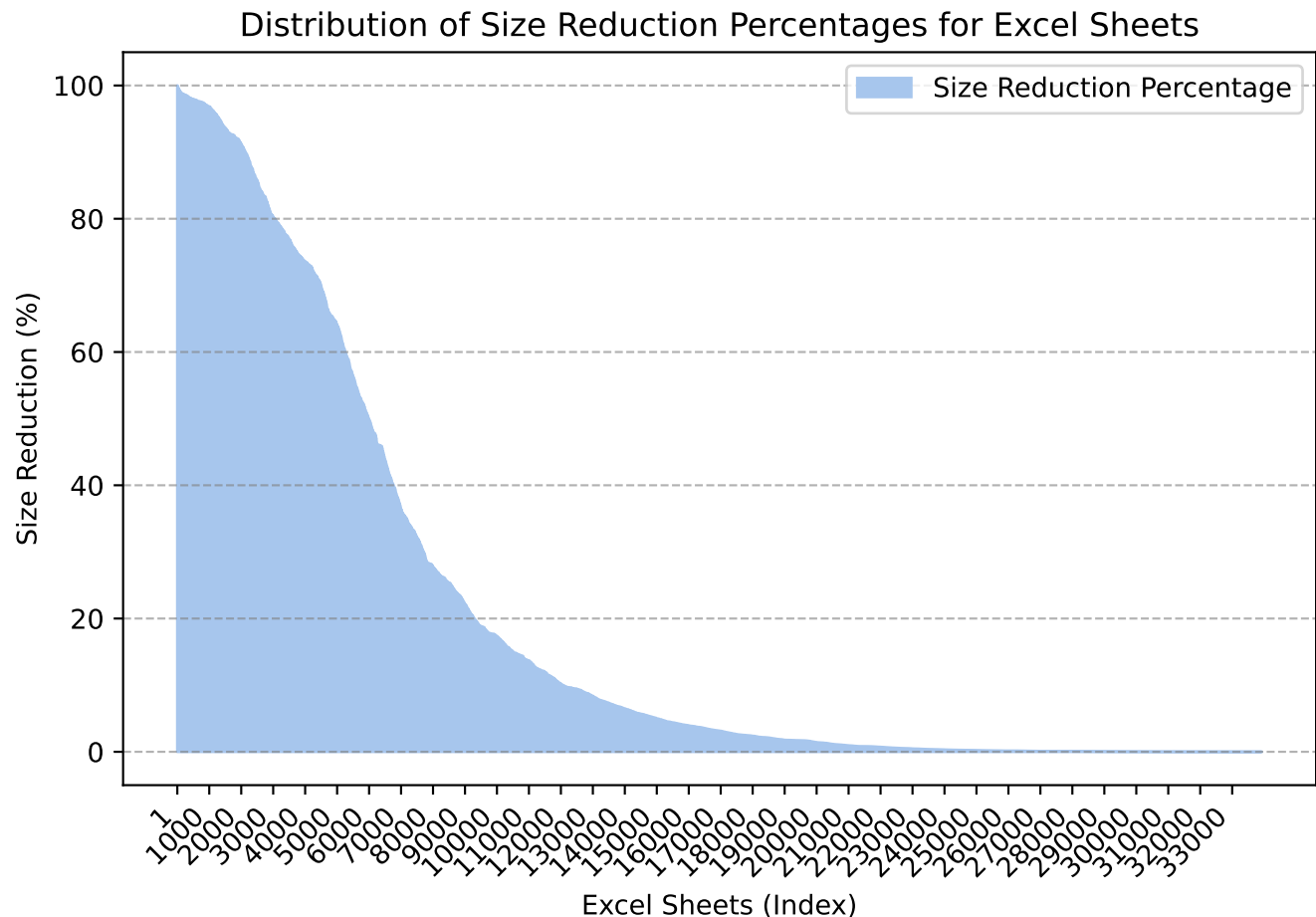


Figure 1: Distribution of Size Reduction Percentages for Excel Sheets

ABSTRACT

The German Federal Statistical Office (Destatis) publishes a multitude of Excel sheets containing statistical data. However, these files are often significantly larger than necessary due to unused cell

formats, empty cells, and other inefficiencies, leading to increased storage requirements and slower data processing. In this paper, we introduce *Excel Shrink*, an open-source tool that reduces the file size of these Excel sheets by up to 99.9%. We analyze the root causes of the excessive file sizes, describe our method for identifying and eliminating redundant content, and compare our tool with existing solutions. Our evaluation on a comprehensive dataset of Destatis Excel files demonstrates substantial improvements in file size reduction without altering the visual appearance of the spreadsheets.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGMOD '25, June 22–27, 2025, Berlin, Germany

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/25/06...\$15.00

<https://doi.org/10.1145/XXXXXXX.XXXXXXX>

CCS CONCEPTS

• **Applied computing** → **Spreadsheets**; • **Information systems** → **Data cleaning**; • **Software and its engineering** → **Extensible Markup Language (XML)**; • **Human-centered computing** → *Graphical user interfaces*; • **Theory of computation** → Data compression.

KEYWORDS

Excel File Optimization, XML Processing, Data Compression, Web Scraping, File Size Reduction, XLSX Format, Data Cleaning, Data Preprocessing, Open Data, Spreadsheet Efficiency, Data Management, Destatis

ACM Reference Format:

Anonymous Author(s). 2025. Excel Shrink: Reducing File Bloat in Destatis Excel Spreadsheets by Up to 99.99%. In *Proceedings of 2025 International Conference on Management of Data (SIGMOD '25)*. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/XXXXXXX.XXXXXXX>

1 INTRODUCTION

Excel files are widely utilized as a format for data exchange across various domains, including finance, statistics, business, and research [?]. Their versatility lies in Excel's capability to store tabular data, perform complex calculations, automate workflows using macros, and visually represent data through charts and graphs. This flexibility makes Excel indispensable in many organizations. However, as spreadsheets grow in complexity, significant challenges arise regarding file size and performance management [?].

Unlike simpler formats such as CSV, which transparently display stored data, Excel files use a more complex structure. Excel employs the Office Open XML format, introduced in Excel 2007, where workbook contents are stored as a collection of XML files within a compressed ZIP archive [?]. This XML-based architecture enables advanced functionality, such as separating data from its presentation layer. For instance, Excel centralizes repeated strings in a shared string table, thus reducing redundancy by storing each text entry once and referencing it elsewhere in the workbook [?]. Furthermore, formatting details—such as fonts, cell styles, and colours—are stored separately from the data itself in dedicated XML files, allowing Excel to reconstruct the sheet's visual appearance when it is opened [?].

1.1 Excel's Internal Storage Mechanisms

To minimize file size, Excel makes extensive use of compression techniques. Once the workbook's data, styles, and structure are defined in XML, these files are bundled into a ZIP archive. The ZIP compression algorithm is particularly effective in this context due to the repetitive nature of XML tags and structures [?]. For example, in a large spreadsheet, tags like `<row>` and `<c>` (for cells) can appear hundreds of thousands of times. The compression algorithm identifies these repetitive patterns and encodes them with shorter representations, reducing the overall file size [?]. Without compression, the verbosity of XML would result in substantially larger files.

Despite achieving considerable space savings, this approach introduces significant inefficiencies. The separation between the visible content seen by the user and the underlying storage structure

can lead to scenarios where redundant or unused data continues to occupy considerable amounts of space in the file. These inefficiencies are not immediately apparent to users but can drastically increase file sizes, reducing the overall performance of both Excel and the systems that interact with it.

One common issue arises when users apply formatting to large ranges of cells that contain no actual data. For example, a user might highlight an entire row or column to change its font style or background colour, but only a subset of the cells are populated with data. The formatting persists across all selected cells, including those that remain empty, creating unnecessary XML records that Excel must store in its underlying files. As Excel stores formatting separately in its XML structure, each formatted but empty cell is treated as a separate entity, adding to the size of the workbook [?]. This problem is exacerbated when users apply conditional formatting or intricate styles over vast areas, further bloating the file without any visible effect.

Similarly, duplicating entire sheets within a workbook can carry over both data and formatting, even when only a portion of the original sheet is needed. Users often duplicate sheets to retain formulas, styles, or layouts, but this also transfers unnecessary metadata and formatting definitions from the original sheet, including empty cells, unused rows, and hidden columns. These redundancies are copied wholesale, and in many cases, users do not realize that they are preserving and propagating unused content. Over time, as sheets are copied and modified, the workbook's file size balloons, with Excel continuing to store and manage a growing list of redundant elements that serve no practical purpose.

A less obvious but equally impactful inefficiency arises from Excel's approach to managing rows and columns. Excel maintains definitions for rows and columns even if they are not populated with data. For instance, if a user inadvertently applies formatting to an entire column but only uses a few cells, the column definition persists in the underlying XML. In some cases, we have observed Excel files where the maximum allowable number of rows and columns were defined, significantly inflating the file size without contributing any value. Although these rows and columns were not fully utilized with data, their definitions were still present in the file, unnecessarily consuming space. These excessive definitions often remain hidden from the user's view, making them difficult to detect and remove manually.

These inefficiencies not only increase storage requirements but also degrade Excel's performance. Larger files take longer to open, save, and modify. In enterprise environments, this can lead to significant delays when handling shared workbooks or automating data processing tasks through scripts and applications that interface with Excel. The excessive size of the files puts a strain on memory and processing resources, leading to a slower user experience and increased frustration in collaborative settings where files are frequently shared or integrated with other systems. These inefficiencies, therefore, represent a considerable challenge for users working with large datasets or managing multiple Excel files.

By addressing these issues—removing redundant formatting, eliminating unused rows and columns, and optimizing the internal XML structure—significant reductions in file size and performance improvements can be achieved. This is the key focus of our open-source tool, `excel-shrink`, which identifies and eliminates these

inefficiencies while preserving the visual and functional integrity of the spreadsheet.

1.2 Impact of File Bloat on Performance

Excel file bloat—characterized by excessive file size without a corresponding increase in useful content—can severely impact performance [?]. Large files take longer to open, save, and process, whether accessed by human users or automated scripts. In enterprise environments, where Excel files are frequently shared, modified, and stored, these performance bottlenecks can disrupt workflows and reduce operational efficiency [?]. Our work with datasets from the German Federal Statistical Office (Destatis) exemplifies these performance challenges. The Excel files we analyzed contained substantial redundant content, including unused rows, columns, and formatting definitions, which contributed to their excessive size and slow processing times [?]. Some of these files exceeded several hundreds of megabytes in their uncompressed form, taking an unusually long time to open and save, both in Excel and through automated means.

1.3 Analyzing Redundant Content in Excel Files

Redundant content in Excel files manifests in various forms, each contributing to file bloat in different ways. The most prevalent types of redundancy include:

- **Empty but formatted cells:** When users apply formatting (such as fonts, colours, or borders) to large ranges of cells, the formatting persists even in cells that contain no data [?]. This results in thousands of empty cells consuming unnecessary space without any visible effect on the sheet's content.
- **Unused rows and columns:** Entire rows or columns may be defined within the workbook's XML structure despite containing no data. These unnecessary definitions contribute to bloating the file size [?].
- **Duplicated sheets:** Users often copy entire sheets for convenience, inadvertently duplicating redundant structures such as empty cells with formatting or hidden rows and columns [?].

To quantify the impact of these redundancies, we performed a detailed analysis of Excel files from Destatis. Our investigation revealed that many files contained vast amounts of poorly structured or unused content that provided no functional benefit. In extreme cases, we discovered that worksheets contained definitions for rows and columns extending to Excel's maximum limits, despite containing no data. By removing these excess elements, we were able to reduce file sizes by as much as 99.99% in some instances.

1.4 Related Work on Excel File Optimization

Several tools and techniques have been developed to mitigate Excel file bloat [?]. These include commercial software, Excel add-ins, and built-in Excel features designed to remove redundant content and reduce file sizes. However, most existing tools have notable limitations. Many are restricted to processing a limited number of files or are constrained to specific Excel versions or enterprise environments [?]. Furthermore, these tools either fail to remove

all redundant content or inadvertently alter the sheet's visual appearance, making them unsuitable for users who need to preserve the layout of their workbooks [?].

These limitations underscored the need for a more robust, scalable solution. To address this gap, we developed our open-source tool, `excel-shrink`, capable of processing large batches of Excel files while maintaining the integrity of the spreadsheet layout.

1.5 Our Contribution: `excel-shrink`

To combat the issues posed by file bloat in Excel, we introduce `excel-shrink`, an open-source tool designed to efficiently reduce the size of Excel files by removing redundant content [?]. The tool conducts an in-depth analysis of the workbook's XML structure to identify and eliminate unnecessary elements, including empty but formatted cells, unused rows, and columns.

Unlike existing tools, `excel-shrink` is capable of processing unlimited numbers of Excel files, making it suitable for handling large datasets, such as the 1,182 files we analyzed from Destatis. The tool ensures that the visual layout of the spreadsheet is preserved, preventing the loss of important formatting. By automating the cleanup of redundant content, `excel-shrink` significantly reduces the time and effort required to optimize Excel files, making it an ideal solution for organizations dealing with large-scale spreadsheets.

In this paper, we present the technical foundations of `excel-shrink`, outline the methodology used to analyze Excel file bloat, and evaluate the tool's performance across various datasets. Our results demonstrate that `excel-shrink` can reduce file sizes by up to 99.99% without compromising the usability or appearance of the Excel files.

2 SCRAPING DESTATIS

The Destatis website does not provide an API for searching publications, making it necessary to interact with the website directly. To automate this process, we developed a web scraper that searches for relevant publications while minimizing the number of requests to the server. Through extensive testing, we discovered that Excel files are exclusively found in the "Publication" document type. By using the GET parameter `documentType=publication`, we were able to significantly narrow down our search results.

Initially, we assumed that searching for the term `xlsx` would identify all publications containing Excel file attachments. However, this search only returned about 100 results, far fewer than the actual number of Excel files available on the Destatis website. Furthermore, there is no option to retrieve a complete list of all publications ever created. To overcome this, we refined our strategy by searching based on publication years. A publication should appear in the search results when queried by its publication year. Although this approach returns some irrelevant results—where the publication year might appear in the text or a file attachment—the number of requests is greatly reduced by caching and reusing links to the Excel file attachments. This ensures that the same files are not downloaded multiple times.

The search results on the Destatis website, like many others, are paginated, with the interface offering the ability to display 10, 12, or 30 entries per page. The number of results per page can be adjusted

using the GET parameter `resultsPerPage=30`. While the interface does not provide an option for more than 30 results per page, we found that manually setting this parameter allowed us to request up to 100 results per page. Through testing, we confirmed that 100 is the maximum number of results that can be returned at once. We set the `resultsPerPage` parameter accordingly to reduce the number of requests further.

Once the results for a page are fetched by the web scraper, we simulate clicking the "next page" link to load subsequent results. The helper function `get_next_page_url` handles this step, using the CSS class `forward button` to identify the link. The URL for the next page is extracted via the XPath expression `//a[@class='forward button']/@href`.

Listing 1: Function to retrieve the link to the next page

```
def get_next_page_url(html: str) -> str:
    xpath = "//a[@class='forward
        button']/@href"
    tree = etree.HTML(html)
    next_page = tree.xpath(xpath)
    if next_page:
        return next_page[0]
    return ""
```

The web scraping and downloading process is outlined in Algorithm 1.

The URL for a subsequent page in the search results looks like this:

`/SiteGlobals/Forms/Suche/Expertensuche_Formular.html?templateQueryString=2000>p=474_list%253D2&documentType_=publication&resultsPerPage=100`

Although the URL and query parameters remain largely the same across requests, one query parameter stands out: `gtp=474_list%253D2`. The `gtp=474_list` portion is consistent, but its meaning is unclear—possibly "get page." The `%253D` is a doubly encoded sequence, where `%253D` is the URL-encoded form of `%3D`, which itself encodes the equal sign (`=`). After decoding, the query parameter becomes `gtp=474_list=2`. The number after the second equal sign corresponds to the current page index in the pagination of the search results.

Once we have queried each publication year from 2000 to 2024, and the file attachments have been extracted from the result pages until no further "next page" button is available, the scraping process for the Excel files is complete.

2.1 Handling Duplicate Filenames

Initially, we assumed that the URLs for the Excel files would be unique, but that the filenames of the downloadable files might not be. Therefore, we added a helper function, `check_for_duplicate_filenames`, to ensure that each file is unique before downloading. Upon running this check, we found no duplicate filenames. Thus, the files can be saved using their original filenames during the download process.

Listing 2: Function to check for duplicate filenames

Algorithm 1 Web Scraping and Downloading Excel Files with Caching

Input: Base URL B, form URL F, download folder D, CSV cache file C, crawl delay (30 seconds)

Initialize: Cached URLs set U, and load file records from CSV C
Create folder D if it does not exist

{— Scraping Process —}

for each year in years **do**

Construct the search URL using year and base parameters

while there are more pages to fetch **do**

Send HTTP GET request to search URL

Parse HTML to extract .xlsx links

Filter links that are not already in the cache U

for each new link **do**

Extract file name from link

Add new link and file name to file records

Append new records to cache C

end for

Save the updated file records back to CSV C

Check for the next page button

if next page exists **then**

Update URL to the next page

Wait for CRAWL_DELAY seconds

else

Exit the loop

end if

end while

end for

{— Checking for Duplicate Filenames —}

Combine cached and new file records

Check if there are duplicate filenames

if duplicates exist **then**

Print warning and exit

end if

{— Download Process —}

for each new file record **do**

Download file from the URL

Save it in the download folder D

Wait for CRAWL_DELAY seconds

end for

```
def check_for_duplicate_filenames(file_records:
    List[FileRecord]) -> bool:
    filenames = set()
    duplicates = set()

    for record in file_records:
        if record.file_name in filenames:
            duplicates.add(record.file_name)
        else:
            filenames.add(record.file_name)

    if duplicates:
        print(f"Duplicate filenames found:
            {duplicates}")
```

```
return True
return False
```

2.2 Respecting Crawl Delay

To minimize the load on the Destatis website, we adhered to the crawl-delay directive specified in the site's robots.txt file, pausing for 30 seconds between requests. This delay ensures that our scraper operates within the guidelines provided by Destatis: Crawl-delay: 30.

3 RELATED WORK

3.1 Inquire

3.2 NXPowerLite Desktop

4 LÖSCHEN VON WHITE SPACES

Der Benutzer von Excel hat keine einfache Möglichkeit herauszufinden, welche Zellen im Arbeitsblatt nur whitespaces enthalten. Um die Whitespaces zu kennzeichnen haben wir im diesem Arbeitsblatt mit 4121 Whitespaces in Abbildung die Zellen mit ausschließlich whitespaces rot gekennzeichnet. [3]

Figure 2

Listing 3: Die Zelle D844 in sheet28.xml

```
<row r="844" spans="1:16" ht="14.1"
  customHeight="1">
  <c r="D844" s="428" t="s">
    <v>102</v>
  </c>
```

Um diese Whitespaces zu erkennen reicht es nicht, die Zelle selbst zu untersuchen denn selbst ein einzelnes leerzeichen wird von Excel in der Datei für wiederkehrende Texte abgelegt, selbst wenn der Text nur genau einmal in der gesamten Excel-Datei vorkommt. Die Zelle D844 sieht im xml folgendermaßen so aus wie in Listing 3.

Die Zeile ist durch das <row>-Tag gekennzeichnet und hat das Attribut r="844", was bedeutet, dass es sich um die 844. Innerhalb der Zeile befindet sich das <c>-Tag, das eine Zelle darstellt. Hier wird die Zelle durch das Attribut r="D844" identifiziert. Das Attribut t="s" gibt den Typ der Zelle an; in diesem Fall steht s für "String", was bedeutet, dass der Inhalt der Zelle als Zeichenkette interpretiert wird.[1]

Der Wert der Zelle wird im <v>-Tag gespeichert, wobei der Wert 102 angegeben ist. Dieser Wert wird als Index in einer String-Tabelle interpretiert, was bedeutet, dass er auf einen bestimmten Text in einer separaten Liste verweist, die in der Excel-Datei definiert ist, nämlich in der sharedStrings.xml Datei. Um den Wert zu finden müssen wir in der sharedStrings.xml Datei nach dem Knoten sst suchen, welcher in Listing 4 zu sehen ist.

Der gegebene XML-Ausschnitt zeigt einen Teil der Shared String Table (SST) in einer Excel-Datei. Die Abkürzung sst steht für *Shared String Table*. Die Shared String Table ist eine Struktur, die es ermöglicht, gemeinsame Zeichenfolgen in einer Excel-Arbeitsmappe zu speichern und zu verwalten. Anstatt jede Zeichenfolge in jeder Zelle separat zu speichern, wird jede eindeutige Zeichenfolge in der Shared String Table einmal abgelegt. Dies führt zu einer Einsparung von Speicherplatz, da die Zellen dann auf diese Zeichenfolgen über Indizes verweisen. Jeder Eintrag in dieser Tabelle hat eine aufsteigende Indexnummer, beginnend mit 0 für die erste Zeichenfolge, 1 für die zweite und so weiter. In diesem Beispiel steht das Element <sst> für die gesamte Tabelle, die in diesem Fall count="17962" und uniqueCount="732" Attribute aufweist.

Listing 4: Die Zelle D844 in sheet28.xml

```
<sst count="17962" uniqueCount="732">
  <si>
    <t>062</t>
  </si>
```

Das Attribut count gibt die Gesamtanzahl der Strings in der Tabelle an, während uniqueCount die Anzahl der eindeutigen Strings repräsentiert. In diesem Fall gibt es 17,962 Strings insgesamt, von denen 732 einzigartig sind. Dies bedeutet, dass viele Strings in der Tabelle mehrfach verwendet werden.

Innerhalb der SST gibt es Elemente vom Typ <si> (Shared Item), die jeweils einen String enthalten. In diesem Beispiel zeigt das Element <t> den tatsächlichen Text des Strings an, der in der Tabelle gespeichert ist. Hier wird die Zeichenfolge 062 dargestellt, die durch den Index in einer Zelle referenziert werden kann.

Um den 102. Index zu finden muss hier nach dem 103. Eintrag gesucht werden. Dieser ist in Listing 5 zu sehen.

Listing 5: Die Zelle D844 in sheet28.xml

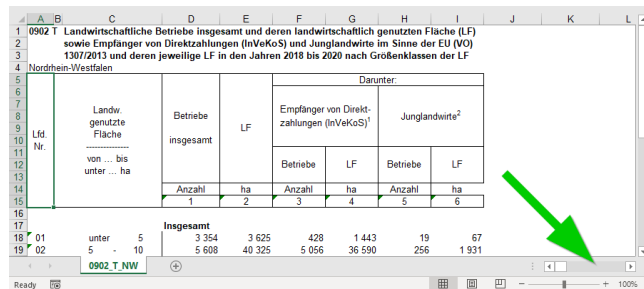
```
<si>
  <t xml:space="preserve"></t>
</si>
```

Innerhalb von <si> befindet sich ein <t> Element, welches den tatsächlichen Text des Strings enthält. In diesem speziellen Fall ist der Inhalt des <t> Elements leer, was bedeutet, dass kein Text gespeichert ist. Das Attribut xml:space="preserve" deutet darauf hin, dass es sich um ein Leerzeichen handelt. Somit können alle Zellen, welche die 102 als Index für den wiederverwendbaren string angeben gelöscht werden, da das Leerzeichen nicht sichtbar ist.

5 LÖSCHEN VON LEEREN SPALTEN

Das erste Anzeichen für zu viele Spaltendefinition kann im horizontalen scrollbar beobachtet werden, wenn man das arbeitsblatt öffnet wie in abbildung 3 zu sehen.[5] Wenn der gesamte bereich

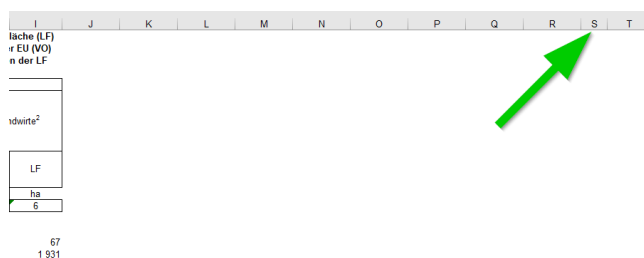
mit den Daten bereits im sichtbaren Bereich ist und trotzdem der anfassers im Scrollbalken sehr schmal und im vergleich zum scrollbaren Bereich sehr klein ist was bedeutet, dass noch eine ganze menge nach rechts gescrollt werden kann, ist das ein Anzeichen dafür, dass eine große Menge an Spaltendefinition das Arbeitsblatt deutlich breiter macht, als es durch den Bereich mit den Daten eigentlich groß ist.



Landwirtschaftliche Betriebe insgesamt und deren landwirtschaftlich genutzte Fläche (LF) sowie Empfänger von Direktzahlungen (InVeKoS) und Junglandwirte im Sinne der EU (VO) 1307/2013 und deren jeweilige LF in den Jahren 2018 bis 2020 nach Größenklassen der LF Nordrhein-Westfalen		Danunter:					
Lfz. Nr.	Landw. genutzte Fläche von ... bis unter ... ha	Betriebe insgesamt	LF	Empfänger von Direktzahlungen (InVeKoS) ¹		Junglandwirte ²	
				Betriebe	LF	Betriebe	LF
		Anzahl	ha	Anzahl	ha	Anzahl	ha
		1	2	3	4	5	6
01	unter 5	3 354	3 625	428	1 443	19	67
02	5 - 10	5 608	40 325	5 056	36 590	256	1 931
Insgesamt							

Figure 3: Sehr breiter Scrollbereich und sehr schmaler scrollbalken

Scrollt man in diesen Bereich, fällt dem benutzer auf, dass zwischen Spalten mit einer einheitlichen breite immer mal wieder spalten auftauchen können die eine davon abweichende Breite haben wie in Abbildung 4 zu sehen.



I	J	K	L	M	N	O	P	Q	R	S	T
fläche (LF) r EU (VO) n der LF											
schwirte?											
LF											
ha											
6											
67											
1 931											

Figure 4: Custom Column Width for a Column outside the Data Section

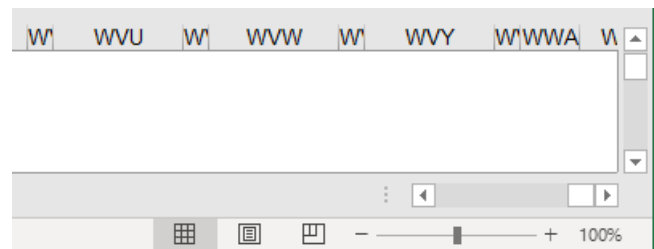
Diese Spaltendefinitionen werden in excel in der sheetX.xml in dem Knoten cols definiert. definiert. In unserem Beispiel sieht die definition der ersten Spalten so aus wie in Listing 6 zu sehen. In diesem XML-Auszug repräsentiert das <cols> Element die Definition von mehreren Spalten in einem Excel-Arbeitsblatt. Jede <col> Deklaration enthält spezifische Informationen zu einer oder mehreren Spalten. Das Attribut min und max gibt den Bereich der Spalten an, den dieses <col> Element umfasst. Zum Beispiel betrifft min="1" max="1" nur Spalte 1, während min="5" max="9" die Spalten 5 bis 9 beschreibt. Das width-Attribut spezifiziert die Breite der Spalten in Excel-Einheiten. Das Attribut customWidth="1" gibt an, dass die Breite der Spalte benutzerdefiniert ist, also vom Benutzer manuell angepasst wurde, anstatt den Standardwert zu verwenden. Die ersten fünf <col> Definitionen beziehen sich auf den Bereich der Daten. Die letzte Definition, min="10" max="18", bezieht sich jedoch bereits auf einen überflüssigen Bereich. Dieser Abschnitt definiert Spalten, die außerhalb des Bereichs der eigentlichen

Daten im Arbeitsblatt liegen, also die Spalten ab der Spalte J, und stellt somit eine unnötige oder ungenutzte Definition dar.

Listing 6: Die ersten Spaltendefinitionen

```
<cols>
  <col min="1" max="1" width="5.21875"
    style="39" customWidth="1" />
  <col min="2" max="2" width="1.77734375"
    style="39" customWidth="1" />
  <col min="3" max="3" width="20"
    style="39" customWidth="1" />
  <col min="4" max="4" width="11.5546875"
    style="39" customWidth="1" />
  <col min="5" max="9" width="10.5546875"
    style="39" customWidth="1" />
  <col min="10" max="18"
    width="11.5546875" style="54"
    customWidth="1" />
```

Während unserer Untersuchungen konnten wir nicht die Frage klären, wie es zu diesen Spaltendefinitionen kommen konnte. Unsere Vermutung ist, dass das Arbeitsblatt dupliziert wurde und das original arbeitsblatt enthielt diese menge an spalten mit den entsprechenden spaltenbreiten. Dieser Hypothese steht aber der fakt gegenüber, dass die spaltenbreiten für individuelle spalten in den von uns untersuchten Dateien bis knapp an die maximale anzahl von 16384 Spalten heranreichen, aber eben auch nur knapp. In Abbildung 5 ist dieser solcher Fall zu sehen. In diesem Arbeitsblatt befinden sich 2403 Spaltendefinitionen und abzüglich 5 tatsächlich genutzten Spaltendefinitionen, bleiben also 2398 überflüssige Definitionen übrig.



W	WU	W	WW	W	WV	WWWA	W

Figure 5: Custom Column Width for a Column outside the Data Section, Source: foerderprogramme-5411206209005 Sheet: 0902_T_NW

Es ist die Spalte WWA zu sehen und rechts davon noch angeschnitten die Spalte WWB, was der 16147. und 16148. Spalte entspricht. Es erscheint unwahrscheinlich dass ein ursprüngliches Arbeitsblatt tatsächlich einmal so viele Spalten verwendet hatte. Er erfordert weitere Forschung herauszufinden, wie solche muster in Excel Dateien entstehen können.

Die letzten in der Abbildung sichtbaren Spalten sind in Listing 7 zu sehen.

Listing 7: Die letzten Spaltendefinitionen im Arbeitsblatt


```
<col min="16146" max="16146"
  width="2.21875" style="39"
  customWidth="1" />
<col min="16147" max="16147"
  width="5.21875" style="39"
  customWidth="1" />
<col min="16148" max="16384"
  width="8.77734375" style="39"
  customWidth="1" />
</cols>
```

Die erste Spalte `min="16146" max="16146"` hat eine festgelegte Breite von 2,21875 und bezieht sich auf Spalte WVZ, welche auch in der Abbildung 5 als die Spalte links neben WWA zu sehen ist jedoch zu schmal ist, um mehr als das "W" zu erkennen. Die zweite Spalte `min="16147" max="16147"` definiert Spalte WWA mit einer Breite von 5,21875. Die letzte Definition `min="16148" max="16384"` deckt einen größeren Bereich von Spalte 16148 bis 16384 ab, also der Spalten WWB bis XFD und weist für diese Spalten eine Breite von 8,77734375 zu. Das erklärt, warum man bei der Spalte WWB landet, wenn man den Scrollbalken nutzt um ganz nach rechts zu scrollen. Die vorletzte Definition definiert, wo der datenbereich endet und damit auch der Scrollbereich endet und die letzte Definition weist lediglich allen übrigen Spalten eine Standardbreite zu. Überraschend ist, dass diese Spaltendefinitionen sich bis in die Unendlichkeit forsetzen würden wie es scheint, nur dass sie an dem von Excel definierten maximalen Limit von 16384 Spalten enden. Wir haben keine Erklärung dafür, wieso kurz vor dem maximalen Limit der maximal zulässigen spalten diese benutzerdefinierten Spaltenbreiten auftauchen.

Um diese Spalten zu löschen müssen folgende Schritte unternommen werden. Das tatsächliche rechte Ende des Bereichs der Daten und Cellformate muss identifiziert werden. Damit steht die erste überflüssige Spaltendefinition fest. Dessen Wert "min" für den Index der ersten Spalte in diesem Bereich von Spalten muss gespeichert werden. Nun können alle überflüssigen Spaltendefinitionen bis auf die letzte gelöscht werden. Der letzten Spaltendefinition wird nun der Wert min überschrieben und zwar mit dem gespeicherten Wert der ersten überflüssigen Spaltendefinition. Die Einhaltung dieser Schritte ist wichtig. Wir haben damit experimentiert lediglich die überschüssigen Spalten zu löschen und nicht dafür zu sorgen dass die letzte Spaltendefinition eine Range bis zur letzten möglichen Spalte 16384 hat. Doch das führte dazu dass Excel diese Arbeitsblätter offenbar als korrupt einstufte und gewisse Reparaturen am Arbeitsblatt vornahm. Teil dieser Änderungen war das Ändern der Standardzeilenhöhe. Das führte dazu dass manche Arbeitsblätter sich leicht optisch änderten. Das ist in Abbildung sichtbar. Auf der linken Seite ist das Original, auf der rechten Seite die von Excel reparierte Version. Einige Zeilen wie etwa die Zeile 6 hat sich durch die Reparatur in der Höhe geändert wodurch es zu sichtbaren Verschiebungen im Arbeitsblatt kommt. Durch das korrekte Löschen von Spaltendefinitionen mit einer letzten Spaltendefinition die bis zum Limit der Spalten bis Spalte 16384 geht geschieht dieser Fehler nicht.

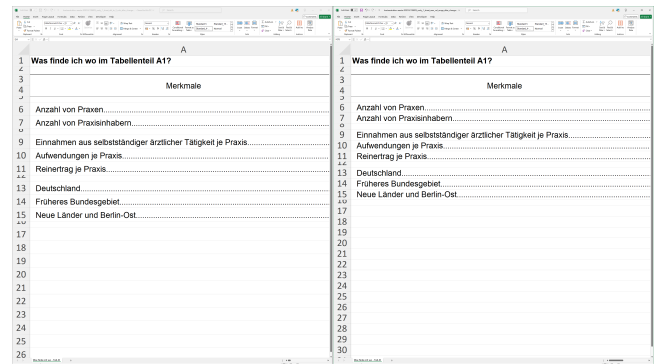


Figure 6: Originale Version mit korrekten Zell-Borders und Zeilenabständen, Source: Sheet: 2_(1)

6 LÖSCHEN LEERER ZELLEN

Einige von unseren untersuchten Arbeitsblättern enthielten riesige Mengen von Zellen ganz ohne Inhalt. Der extremste Fall war ein Arbeitsblatt 8,141 Zellen mit Wert und 5237075 Zellen ohne Wert, dementsprechend hatten nur 0.15 % der gespeicherten Zellen tatsächlich einen Wert. [4][Sheet "csv-71717-25"]

zeigt die letzte Zeile dieses Arbeitsblattes.

In dem Listing 8 sehen wir eine Darstellung eines Zeilen-Elements in diesem Excel-Arbeitsblatt, das speziell die letzte mögliche Zeile in Excel anzeigt, nämlich die Zeile 1.048.576. Das `<row>`-Element enthält mehrere Attribute: Das Attribut `r` gibt die Zeilennummer an. Hier zeigt `r="1048576"`, dass es sich um die maximale Zeilenanzahl in Excel handelt. Das Attribut `spans` spezifiziert den Bereich der Spalten, den die Zeile abdeckt. In diesem Fall bedeutet `spans="3:7"`, dass die Zeile die Spalten C bis G überspannt. Das trifft auch für die 1,047,407 darüber bis zur Zeile 1169 zu. Innerhalb des `<row>`-Elements gibt es mehrere `<c>`-Elemente (Zellen). Jedes `<c>`-Element hat eigene Attribute: Das Attribut `r` gibt den Zellbezug an. Zum Beispiel bezieht sich `r="C1048576"` auf die Zelle in Spalte C der Zeile 1.048.576.

Listing 8: Die letzte Zeile im Arbeitsblatt csv-71717-25

```
<row r="1048576" spans="3:7" ht="12.75"
  customHeight="1">
  <c r="C1048576" s="538"/>
  <c r="D1048576" s="537"/>
  <c r="E1048576" s="537"/>
  <c r="F1048576" s="537"/>
  <c r="G1048576" s="541"/>
</row>
</sheetData>
```

Anders als noch bei den Whitespaces enthalten diese Zellen nun gar keinen Wert und müssen daher auch nicht mit der shared-String datei abgeglichen werden. Allerdings kann es sein dass solche leeren Zellen ohne jeglichen Inhalt immerhin eine Hintergrundfarbe zugewiesen wurde, wie es bei einigen Deckblättern zum Gestalten der Seite der Fall ist oder für das Hervorheben von Tabellenköpfen. Andererseits können diesen Zellen auch border hinterlegt sein.

Um das herauszufinden muss man sich das Attribut `s` ansehen, denn es steht für den Stilindex, der auf die Zelle angewendet wird. Die Werte `s="538"`, `s="537"` und `s="541"` stehen dabei für einen Index für eine lookuptable in der alle Zell-Styles abgelegt sind. Die zu den Stilindex passenden Styles findet man in der `xl/styles.xml` Datei. Die entsprechenden Ausschnitte sind in Listing 9 zu sehen.

Listing 9: Die letzte Zeile im Arbeitsblatt csv-71717-25

```
<cellxf count="778">
  <!--538-->
  <xf numFmtId="0" fontId="0" fillId="0"
    borderId="0" xfId="0"/>
  <!--539-->
  <xf numFmtId="0" fontId="4" fillId="0"
    borderId="0" xfId="3"
    applyNumberFormat="1" applyFont="1"/>
  <!--542-->
  <xf numFmtId="0" fontId="1" fillId="0"
    borderId="0" xfId="1"
    applyNumberFormat="1" applyFont="1"
    applyAlignment="1">
    <alignment horizontal="right"/>
  </xf>
```

Die XML-Struktur innerhalb von `<cellxf>` enthält Formatierungseigenschaften, die für Zellen in einer Excel-Datei definiert sind. Jedes `<xf>`-Element stellt eine Formatierungsregel dar. Genau wie bei der Shared Strings datei werden die Formatierungsregeln über ihre Reihenfolge in der Liste indiziert, beginnend mit der ersten bei 0 und dem zweiten bei 1, und so weiter. Für die ids `s="538"`, `s="537"` und `s="541"` muss also nach dem 538., dem 539. sowie dem 542 Element in der Liste gesucht werden. Die Attribute steuern das Erscheinungsbild der Zelle. Das Attribut `numFmtId` gibt das Zahlenformat an, das für die Zelle verwendet wird, während `fontId` die ID der Schriftart beschreibt. `fillId` bezieht sich auf die Füllfarbe, und `borderId` definiert die Randeigenschaften. `xfId` verweist auf eine Formatvorlage, von der die Zelle Eigenschaften erbt. Die Attribute `applyNumberFormat`, `applyFont` und `applyAlignment` legen fest, ob das entsprechende Zahlenformat, die Schriftart und die Textausrichtung auf die Zelle angewendet werden. Falls das Element `alignment` vorhanden ist, spezifiziert es die Ausrichtung des Textes innerhalb der Zelle, wie z.B. eine rechtsbündige Ausrichtung. Diese Struktur gibt die Formatierungsregeln an, die für die Darstellung und das Layout der Zelleninhalte verwendet werden.

An diesen Styles lässt sich überprüfen, dass diese Zellen lediglich styles anwenden, die bei einer leeren Zelle nicht sichtbar sind. Sie verwenden unterschiedliche Schriftarten, Zelle G1048576 ist zudem rechtsbündig formatiert. Alle diese Zellen sind demnach überflüssig und können dementsprechend gelöscht werden.

Ein anderer Fall ist es, wenn die Formatregeln `borders` enthalten. Werden diesen Zellen einfach so gelöscht passiert, was uns in früheren Versionen des Scriptes passiert ist. Dies ist in Abbildungen 7b und 7a zu sehen. [2][Sheet: 2_(1)] In Abbildung 7b sind alle Umrandungen um die Zellen der Tabellen-Header. In Abbildung 7a sind nur die Umrandungen sichtbar, die an Zellen mit Inhalt grenzen. So fehlt beispielsweise der Border zwischen den Zellen A2, A3.

	A	B	C	D	E	F	G	H	I	J	K	L
1	Straßenverkehrsunfälle, Verunglückte, Bestand an motorisierten Fahrzeugen und Bevölkerung 1906 – 2021											
2		Polizeilich erfasste Unfälle ¹	Verunglückte ^{1,2}		Bestand an motorisierten Fahrzeugen ³		Bevölkerung ⁷					
3		insgesamt	davon mit	insgesamt	davon	insgesamt	darunter	insgesamt	darunter	insgesamt	darunter	
4			Persone-	Sach-		Geladete	Verletzte		Motor-	Persone-	Güterkraft-	
5			schaden	schaden					räder mit	kraftwagen ⁵	fahrzeuge ⁶	
6									amt. Kenn-			
7									zeichen ⁴			
8												
9		Anzahl										1 000
10		Reichsgebiet ⁸										
11	1906	2 290	1 303	987	1 570	51	1 519					
12	1907	4 864	2 230	2 634	2 564	145	2 419	27 026	15 954	10 115	957	
13	1908	5 069	2 342	2 727	2 771	141	2 630	36 022	19 808	14 671	1 543	

(a) Originale Version mit korrekten Zell-Borders und Abständen

	A	B	C	D	E	F	G	H	I	J	K	L
1	Straßenverkehrsunfälle, Verunglückte, Bestand an motorisierten Fahrzeugen und Bevölkerung 1906 – 2021											
2		Polizeilich erfasste Unfälle ¹	Verunglückte ^{1,2}		Bestand an motorisierten Fahrzeugen ³		Bevölkerung ⁷					
3		insgesamt	davon mit	insgesamt	davon	insgesamt	darunter	insgesamt	darunter	insgesamt	darunter	
4			Persone-	Sach-		Geladete	Verletzte		Motor-	Persone-	Güterkraft-	
5			schaden	schaden					räder mit	kraftwagen ⁵	fahrzeuge ⁶	
6									amt. Kenn-			
7									zeichen ⁴			
8												
9		Anzahl										1 000
10		Reichsgebiet ⁸										
11	1906	2 290	1 303	987	1 570	51	1 519					
12	1907	4 864	2 230	2 634	2 564	145	2 419	27 026	15 954	10 115	957	
13	1908	5 069	2 342	2 727	2 771	141	2 630	36 022	19 808	14 671	1 543	

(b) Korrupte Version entfernten Borders und Abständen

Figure 7: Fehlerhafte Entfernung von Zellen mit Zell-Borders und Löschung von Zeilen mit benutzerdefinierter Höhe

Die Zelle A2 sieht im XML aus, wie in Listing 10 zu sehen.

Listing 10: Die Zelle A2 in der sheet7.xml

```
<c r="A2" s="577"/>
```

Hier ist die style id `s="577"`. Er definiert den für die Zelle zugrundeliegende Zellformatierung, welche wiederum die id für die border-definition enthält.

In der `styles.xml` Datei unter dem Knoten `cellxf` suchen wir also den `xf` Knoten mit der id 577, also den 578. `xf` Knoten. Dieser ist in Listing 11 zu sehen.

Listing 11: Style mit der impliziten id 577 in der style.xml

```
<xf numFmtId="0" fontId="8" fillId="2"
  borderId="5" xfId="0" applyFont="1"
  applyFill="1" applyBorder="1"
  applyAlignment="1"/>
```

Dieser `xf` knoten hat für das Attribut `borderId` den Index 5 gesetzt. Genau so wie auch schon für die Zell Styles bedeutet dieser Index, dass der 6. `border` Knoten in dem Knoten `borders` in derselben `styles` Datei gemeint ist.

Listing 12 zeigt den 6. `border` Knoten mit der impliziten id 5.

Listing 12: Der border Knoten mit der impliziten id 5 in der style.xml Datei

```
<borders count="12">
  <!-- ... -->
  <border>
    <left/>
    <right/>
    <top/>
    <bottom style="thin">
      <color indexed="64"/>
    </bottom>
    <diagonal/>
  </border>
```


Jeder border knoten hat die unterknoten left, right, top, bottom und diagonal. Wenn diese Knoten keine Attribute bzw keine unterknoten haben, dann bedeutet dies, das die Zelle keine Border hat. Dort hat der Knoten bottom das attribut style mit dem wert "thin". weiterhin hat der knoten den unterknoten color mit dem attribut indexed und dem wert "64". Das bedeutet, dass die zugeordnete Zelle eine dünne Linie unten mit der gleichen Farbe hat, welcher der Index 64 zugeordnet ist. In diesem Fall ist das die Farbe schwarz. Zellen, für die über diesen Weg herausgefunden wird, dass sie ein border haben, dürfen nicht gelöscht werden. Nur wenn eine Zelle sowohl keine Inhalt hat, als auch einen border knoten zugewiesen hat, dessen Knoten alle keine attribute haben, um anzuzeigen, dass sie kein Border haben, dürfen gelöscht werden.

7 LÖSCHEN LEERER ZEILEN

Ähnliches wie für die Zellen gilt auch für die Zeilen. Wenn eine Zeile keine definierte Zellen hat, kann es immernoc sein, dass die Zeile eine definierte Höhe hat. Wir die Zeile gelöscht, passiert das, was in Abbildung 7a im Vergleich zum Original 7b zu sehen ist. Der Abstand zwischen der Überschrift und der taele schrumpft, da die zweite Zeile, die als Abstand zwischen Überschrift und Tabelle diene gelöscht ist und für diese Zeile die Standardhöhe für Zeilen in diesem Arbeitsblattes angewendet wird, welche kleiner ist, als die benutzerdefinierte Höhe der Zeile.

Um sicherzugehen, dass wir beim löschen von leeren Zeilen die optische representation des arbeitsblattes nicht ändern, wählen wir eine ähnliche strategie wie auch schon bei dem entfernen der Columns. Wir finde zuvor raus in welcher range tatsächlich Werte in den Zellen oder Hintergrundfarben oder borders definiert sind. Danach gehen wir durch alle leere Zeilen und löschen sie nur in dem Fall, dass sie außerhalb dieses bereiches sind. Auf diese weise können wir die Dateigröße mancher Arbeitsblätter erheblich verringern ohne die Inhalte oder die darstellung der arbeitsblätter zu verändern.

```
<row r="2" spans="1:12" ht="8.25"
  customHeight="1">
\label{fig:custom_height}
```

8 HERRAUSFINDEN DES SHEET NAMENS

Um eine auflisting der verringerten Dateigrößen ausgeben zu können, müssen die die Arbeitsblatttitel herauszufinden. Durch die Arbeitsblattdateien wissen wir allerdings nur eine id, welche im Dateinamen steht, wie etwa sheet1.xml, sheet2.xml und so weiter. Welche sheetnummer zu welchem Arbeitsblatttitel gehört, können wir leider den Titel nicht aus der Sheet Datei ermitteln, denn der Titel wird nur in der xl/workbook.xml den einzelnen sheets zugeordnet. Diese Zuordnung ist in Listing zu sehen. [5] Das Attribut sheetId ist dabei zu ignorieren, die id, welche den Sheet Titel zu der entsprechenden sheetX.xml zuordnet ist das Attribut r:id. Da am Anfang von jedem Wert in diesem Attribut ein rld vorkommt, muss die id extrahiert werden, indem die zeichen beginnend mit dem 4. buchstaben beginnend extrahiert deren. Über die Ids gibt es dann eine eindeutige Zuordnung zu den Arbeitsblatttiteln, die für die ausgabe genutzt werden können.

Listing 13: workbook.xml mit der zuordnung der id zu den Arbeitsblattiteln

```
<sheets>
  <sheet name="Titelseite" sheetId="10"
    r:id="rId1"/>
  <sheet name="Inhalt" sheetId="11"
    r:id="rId2"/>
  <sheet name="Erläuterungen"
    sheetId="3" r:id="rId3"/>
  <sheet name="Allgemeines" sheetId="4"
    r:id="rId4"/>
  <sheet name="0901_T_D" sheetId="12"
    r:id="rId5"/>
  <sheet name="0901_T_BW" sheetId="13"
    r:id="rId6"/>
```

9 EVALUATION

Wir haben unser Excel Shrink Programm auf alle mit dem web scraper heruntergeladenen Excel Dateien von Destatis angewendet. Die Abbildung 8 zeigt die Verteilung der reduktionsrate in prozent für die einzelnen Arbeitsblätter an. Das Diagramm erweckt den Eindruck, dass einige Arbeitsblätter, knapp 100 % in ihrer Dateigröße verringert wurden. Das ist korrekt, denn bei 17 Arbeitsblättern war die Reduktion in Prozent über 99,9 % und bei 95 Arbeitsblättern war die Reduktion in Prozent über 99 % und bei 453 Arbeitsblättern war die Reduktion in Prozent über 98 %.

Table 1: Top Size Reduction Results

verkehrsunfaelle-zeitreihen-xlsx-5462403.xlsx	Technis
verkehrsunfaelle-zeitreihen-xlsx-5462403.xlsx	3.1_(7)
verkehrsunfaelle-zeitreihen-xlsx-5462403 (NXPowerLite Copy).xlsx	3.1_(1)
verkehrsunfaelle-zeitreihen-xlsx-5462403.xlsx	3.1_(1)
verkehrsunfaelle-zeitreihen-xlsx-5462403.xlsx	4.6_(3)
verkehrsunfaelle-zeitreihen-xlsx-5462403.xlsx	5.1.4_(2)
verkehrsunfaelle-zeitreihen-xlsx-5462403 (NXPowerLite Copy).xlsx	5.1.4_(2)
verkehrsunfaelle-zeitreihen-xlsx-5462403.xlsx	10.1
verkehrsunfaelle-zeitreihen-xlsx-5462403.xlsx	10.3
statistischer-bericht-kostenstruktur-med-bereich-2020161227005.xlsx	52571-1

9.1 Datei-Größe

9.2 Dauer die Datei zu öffnen

10 FLUTTER FRONTEND FÜR EASE OF USE

11 CONCLUSION

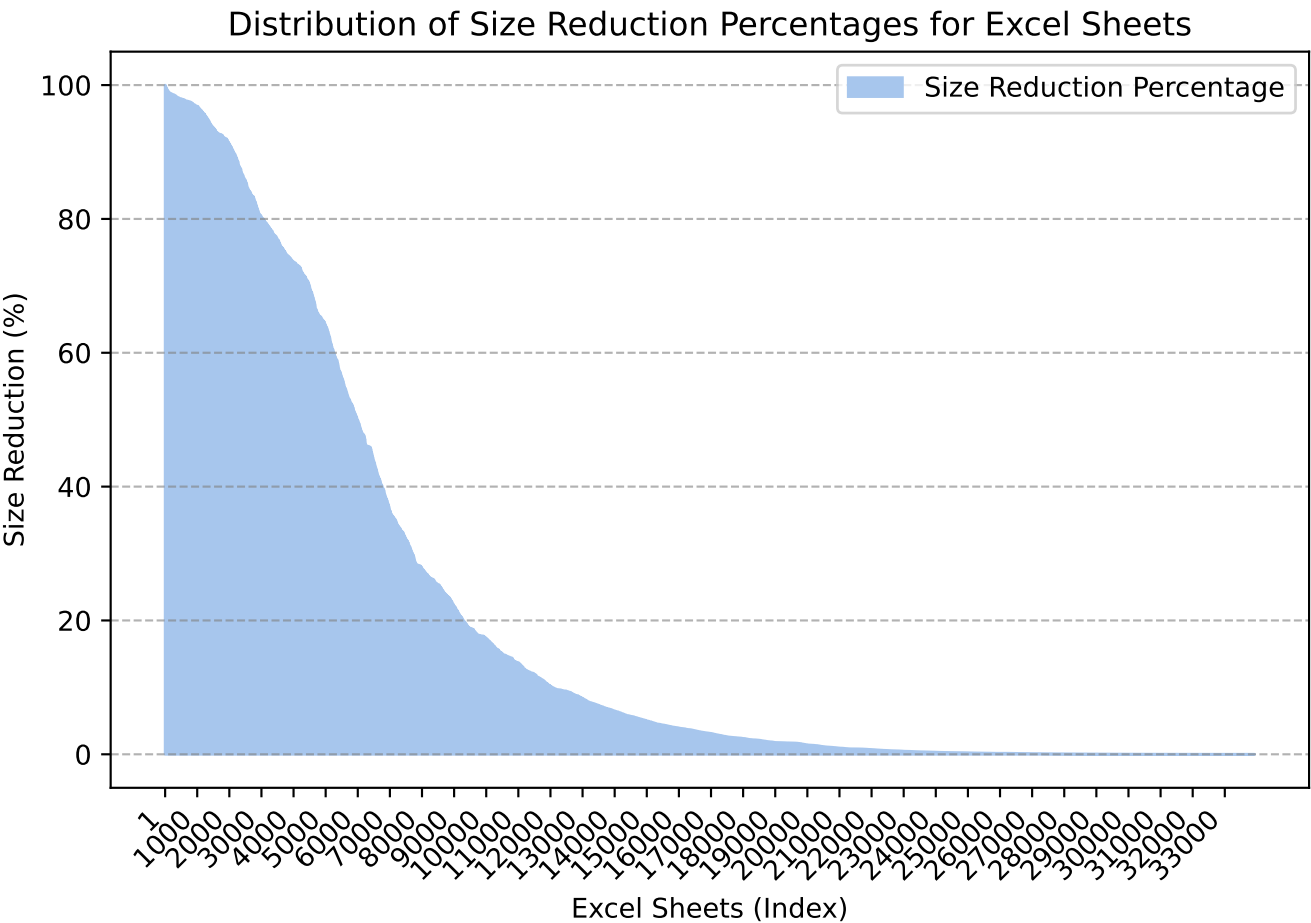


Figure 8: Distribution of Size Reduction Percentages for Excel Sheets

12 RELATED WORK

Wir haben zwei Programme gefundenen welche auch Excel Dateien verkleinern.

Das eine Programm ist ein kommerzielles Programm wofür eine trailversion existiert , namens XXX.

Ein Anderes Programm ist ein Add IN für excel, welches kostenfrei erhältlis ist. Es heißt YYY.

Wir haben beide programme ausprobiert. Im Folgenden gehen wir auf beide programme ein und vergleichen die Ergebnisse mit unserem Programm.

12.1 XXX

Das Programm XXX ist ein kommerzielles Programm. Es existiert eine Trail-Version von ZZZ Tagen.

Das Programm hat im vergleich zu unserem Programm folgende überflüssige Spalten und Zellen übrig gelassen.

12.2 Inquire Add-in

Inquire ist ein Add in für Excel. Das Add-In ist nur für Excel in den Versionen Office Professional Plus und Microsoft 365 Apps

for Enterprise verfügbar. Office Home, Microsoft 365 Family bzw. Single sowie Microsoft 365 Business sind davon ausgenommen.

Das Addin kann aktiviert werden, indem man folgende Schritte ausführt. Man geht über den Menüpunkt File und wählt dann Options. Daraufhin öffnet sich ein Dialog, in welchem man auf auf Add-ins klickt.

Im Dropdown Manage wählt man COM Add-ins wie in Abbildung 9 zu sehen. und klickt anschließend auf "Go ...".

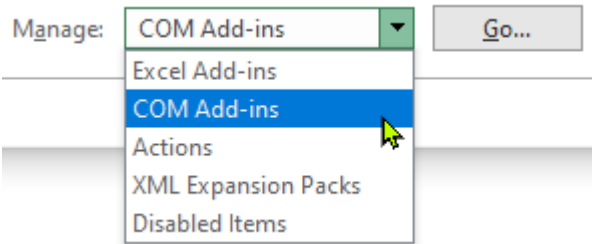


Figure 9: bla bla

Im Daraufhin sich öffnenden Dialog kann das Add-in Inquire aktiviert werden, wie in Abbildung 10 zu sehen ist. Taucht in dem Dialog das Inquire Add-in nicht auf, dann liegt das daran, dass die falsche Version von Excel verwendet wird.

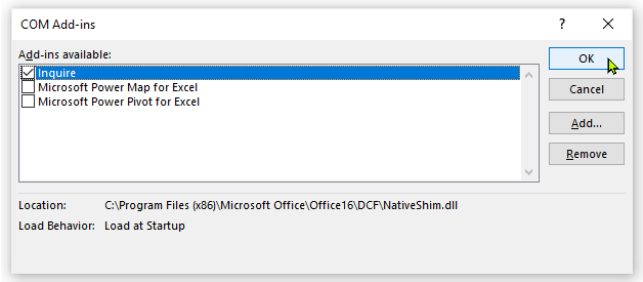


Figure 10: bla bla

Das Programm hat im Vergleich zu unserem Programm folgende überflüssige Spalten und Zellen übrig gelassen:

Das Programm hat keine Batchfunktion. Es können also nicht mehrere Dateien auf einmal bearbeitet werden. Stattdessen muss jede einzelne Datei einzeln geöffnet werden, was vor dem Hintergrund, dass besonders die sehr großen Dateien, die man ja gerne verkleinern möchte, eine lange Zeit von wenigstens einer Minute zum Öffnen benötigen. Mehrere Dateien zu öffnen, die besonders groß sind und nach dem Laden das Inquire Add-in zu benutzen um die Datei zu verkleinern ist damit ein nicht zu unterschätzender Zeitaufwand.

Wir haben die Ergebnisse von Inquire mit unseren Ergebnissen von Excel Shrink anhand von besonders großen Excel-Dateien verglichen.

Inquire lieferte bereits sehr gute Ergebnisse. In manchen Fällen jedoch wurden immens große Arbeitsblätter nicht bereinigt. In einem Arbeitsblatt, welches als Deckblatt verwendet wurde,

waren beispielsweise nur 77 Zeilen genutzt und alle Zeilen ab 78 waren versteckte Zeilen, wie in Abbildung 11 zu sehen.

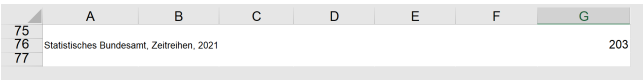


Figure 11: bla bla

Im XML waren aber die Zeilen 78 bis 65533 jeweils mit den Zellen A bis F definiert wie in Listing 14 zu sehen.

Listing 14: Die Zeile 65533 in im sheet170.xml in der Datei verkehrsunfaelle-zeitreihen-xlsx-5462403

```
<row r="65533" spans="1:7" ht="0"
  hidden="1" customHeight="1">
  <c r="A65533" s="68" />
  <c r="B65533" s="68" />
  <c r="C65533" s="68" />
  <c r="D65533" s="68" />
  <c r="E65533" s="68" />
  <c r="F65533" s="68" />
```

```
<c r="G65533" s="68" />
</row>
</sheetData>
```

Bei Anwenden des Inquire Add-in wurden die Zeilen für jedes einzelne Arbeitsblatt

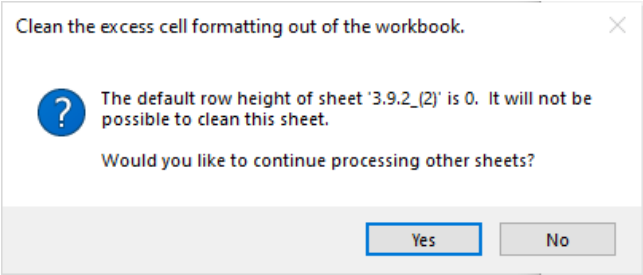


Figure 12: bla bla

verkehrsunfaelle-zeitreihen-xlsx-5462403.xlsx

12.3 Excel Performance Tab

Seit 2022 gibt es für Excel for Web und seit 2023 für Excel für Windows gibt es eine neue Funktion namens Check Performance, welcher im Ribbon Menü über den "Review" Reiter aufgerufen werden kann. Diese Funktion hatte in unserem Vergleich zu den Alternativen die größte Verringerung der Datei-Größe erreicht, schaffte es aber nicht überflüssige Column-Definitionen zu entfernen.

Weiterhin wurden von dieser Funktion auch Zellen entfernt, welche das optische Erscheinungsbild von Arbeitsblättern verändert. So wurde beispielsweise das Deckblatt, welches in Abbildung 13a verändert, sodass die Zellen, die für Weißraum im Deckblatt sorgen entfernen wie in Abbildung 13b.[2]



(a) bla bla

(b) bla bla bla

Figure 13: Gesamtbeschreibung für beide Bilder

Für ein Deckblatt mag das okay sein, aber für Formulare wie das in Abbildung 15, welche aus mehreren Zeilen ohne Inhalt bestehen, da diese ausgedruckt werden sollen um dann ausgefüllt zu werden, ist das ein Problem. [4]

Denn die Check Performance Funktion von Excel entfernt all diese Borders, da sie außerhalb von dem durch die Werte aufgespannten Rechteck sind, wie in Abbildung 16 zu sehen.

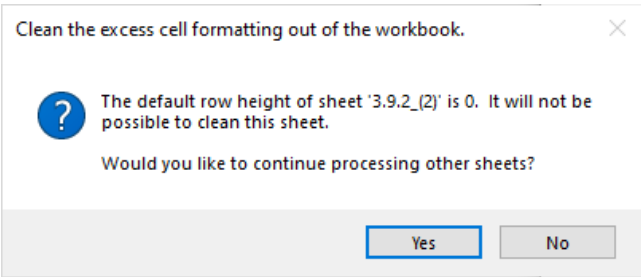


Figure 14: bla bla

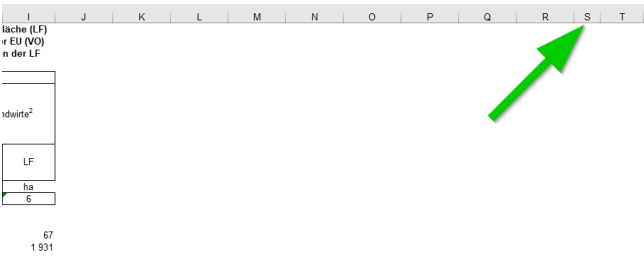


Figure 17: bla bla

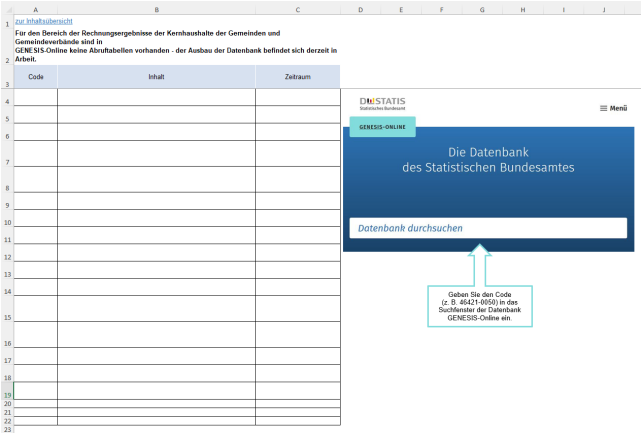


Figure 15: statistischer-bericht-rechnungsergebnis-kernhaushalt-gemeinden-2140331217005 Sheet: GENESIS-Online

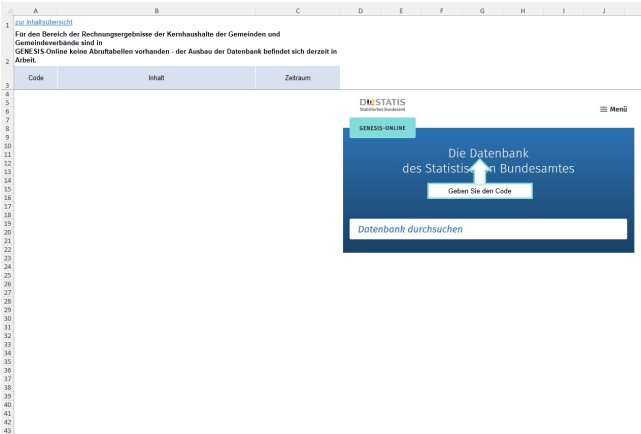


Figure 16: statistischer-bericht-rechnungsergebnis-kernhaushalt-gemeinden-2140331217005 Sheet: GENESIS-Online

12.4 Ergebnisse

Für die größten Dateien haben wir die Ergebnisse von XXX und YYY mit unserem excel shrink verglichen. Siehe Abbildung 17.

13 CONCLUSION

We have presented *Excel Shrink*, an open-source tool that significantly reduces the file sizes of Excel sheets from Destatis by eliminating unnecessary content. Our analysis identified common practices that lead to bloated Excel files and demonstrated how targeted removal of redundant data can achieve reductions of up to 99.9% without altering the spreadsheets' appearance.

Our tool outperforms existing solutions in both file size reduction and preservation of formatting. By making *Excel Shrink* available to the community, we hope to facilitate more efficient data processing and analysis of large-scale Excel datasets.

REFERENCES

[1] Microsoft. 2014. *Open XML SDK 2.5 for Office*. Microsoft Press.

[2] Statistisches Bundesamt Destatis (Federal Statistical Office). 2022. Verkehrsunfälle - Zeitreihen 2021 (Traffic Accidents - Time Series 2021) (Letzte Ausgabe - berichtsweise eingestellt). <https://www.destatis.de/DE/Themen/Gesellschaft-Umwelt/Verkehrsunfaelle/Publikationen/Downloads-Verkehrsunfaelle/verkehrsunfaelle-zeitreihen-pdf-5462403.html>. Accessed on October 7, 2024.

[3] Statistisches Bundesamt Destatis (Federal Statistical Office). 2023. Private Konsumausgaben und Verfügbares Einkommen - 1. Vierteljahr 2023 (Private Consumption Expenditures and Disposable Income - 1st Quarter 2023) (Letzte Ausgabe - berichtsweise eingestellt). <https://www.destatis.de/DE/Themen/Wirtschaft/Volkswirtschaftliche-Gesamtrechnungen-Inlandsprodukt/Publikationen/Downloads-Inlandsprodukt/konsumausgaben-pdf-5811109.html>. Accessed on October 7, 2024.

[4] Statistisches Bundesamt Destatis (Federal Statistical Office). 2023. Statistischer Bericht - Rechnungsergebnisse der Kernhaushalte der Gemeinden und Gemeindeverbände 2021 (Statistical Report - Financial Results of the Core Budgets of Municipalities and Municipal Associations 2021). <https://www.destatis.de/DE/Themen/Staat/Oeffentliche-Finanzen/Ausgaben-Einnahmen/Publikationen/Downloads-Ausgaben-und-Einnahmen/statistischer-bericht-rechnungsergebnis-kernhaushalt-gemeinden-2140331217005.html>. Accessed on October 7, 2024.

[5] Statistisches Bundesamt Destatis (Federal Statistical Office). 2024. Statistischer Bericht - Landwirtschaftliche Betriebe - Förderprogramme - 2023 (Statistical Report - Agricultural Enterprises - Funding Programs - 2023). <https://www.destatis.de/DE/Themen/Branchen-Unternehmen/Landwirtschaft-Forstwirtschaft-Fischerei/Landwirtschaftliche-Betriebe/Publikationen/Downloads-Landwirtschaftliche-Betriebe/statistischer-bericht-foerderprogramme-5411206239005.html>. Accessed on October 7, 2024.

A APPENDIX 1

Received 17 October 2024; revised ; accepted